



Cryptographic Protocol Verification and Analysis

University of Texas at Dallas - Team 1



Team Members

Student Investigators

Manasa Kandala

Pragathi Vemulapalli

Rahul Muthyam

Saloni Shivdasani

Shreyans Patel

Technical Director

Dr. Ed Ziegler

External Support

Enis Golaszewski

Faculty Supervisor

Dr. Brian Ricks



Agenda

- Recap
- Work Done
- Analysis
- Next Steps
- Conclusion
- Q&A



Recap

In this project, we aim to:

1. Study Grant Authorization and Negotiation Protocol (GNAP)
2. Learn Cryptographic Protocol Shapes Analyzer (CPSA)
3. Use CPSA to analyze GNAP
4. Find potential solutions to any vulnerabilities (if found)



Work Done

- Analyze GNAP sequences that involve authentication and model them in CPSA
 - Software-only Authorization
 - User-Code Interaction
- Study the results and devise solutions if vulnerabilities are found



Grant Negotiation and Authorization Protocol (GNAP)

- GNAP is the next version of the OAuth2.x family of protocols
- It boasts several advantages over OAuth2
- Being relatively new and not yet standardized, GNAP is an excellent choice for formal analysis



GNAP Actors

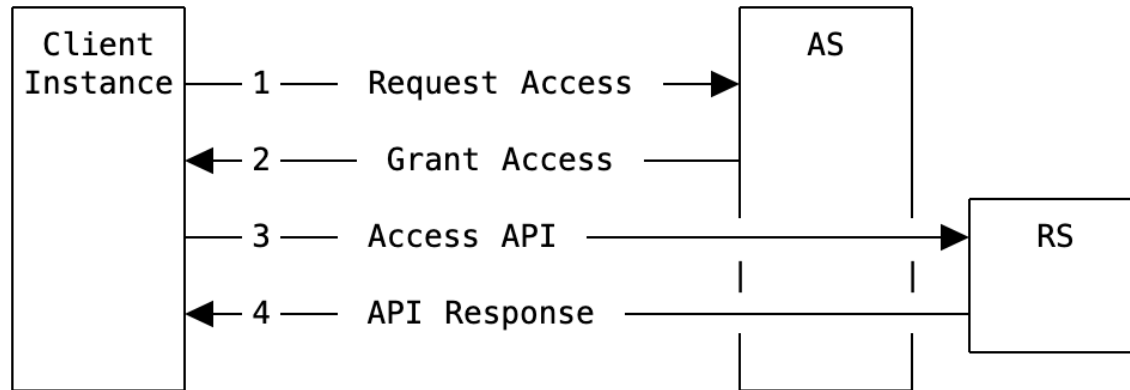
1. (AS) Authorization Server
2. (RS) Resource Server
3. (RO) Resource Owner
4. End User
5. Client Instance



GNAP Assumptions

- Communication channels are secure.
- No party is compromised.

Software-only Authorization





Request Access (1)

Parameters Required:

- Client Instance Key (ci_key)
- Resource Server Id (rs)
- Resource Server Permission (rs_access)



Grant Access (2)

Parameters Required:

- Access Token:
 - Client Instance Key (ci_key)
 - Resource Server Id (rs)
 - Resource Server Permission (rs_access)
 - Digital Signature:
 - Private Key of Authorization Server (invk pk_as)



Access API (3)

Parameters Required:

- Access Token:
 - Client Instance Key (ci_key)
 - Resource Server Id (rs)
 - Resource Server Permission (rs_access)
 - Digital Signature:
 - Private Key of Authorization Server (pk_as)



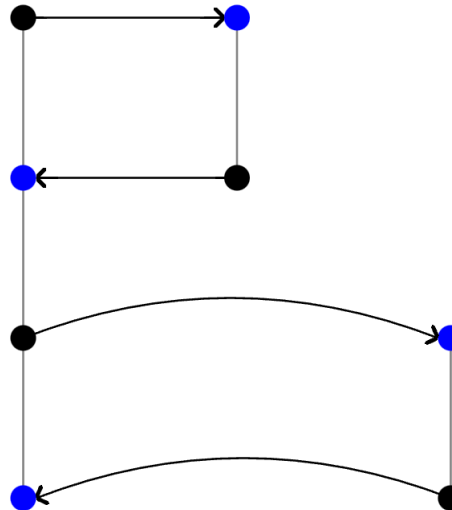
API Response (4)

Parameters Required:

- Data (rs_data)
- Hash (Data + Access Token)

Analysis (Client Instance Perspective)

client_instance authorization_server resource_server



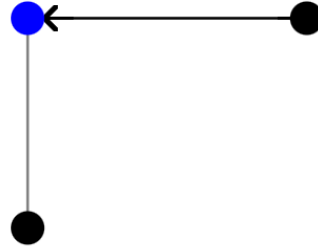
● = Transmission

● = Reception, explained

● = Reception, unexplained 14

Analysis (Authorization Server Perspective)

authorization_server client_instance



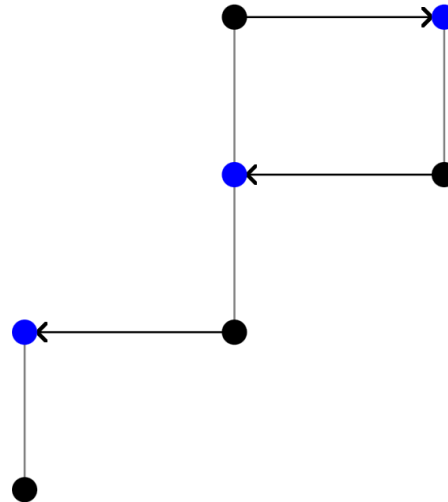
● = Transmission

● = Reception, explained

● = Reception, unexplained 15

Analysis (Resource Server Perspective)

resource_server client_instance authorization_server

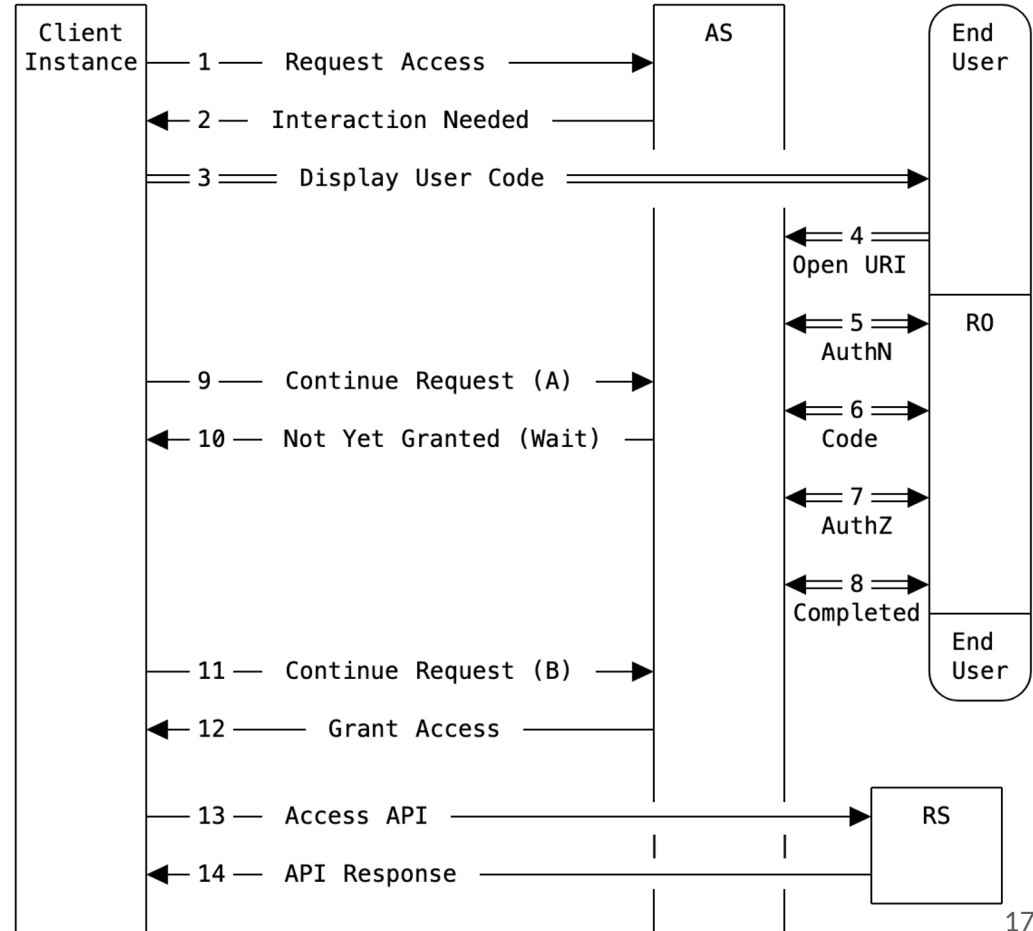


● = Transmission

● = Reception, explained

● = Reception, unexplained 16

User-Code Interaction





Request Access (1)

Parameters Required:

- Client Instance Key (pk_ci)
- Resource Server Id (rs)
- Resource Server Permission (rs_access)
- Digital Signature:
 - Private Key of Client Instance (invk pk_ci)



Interaction Needed (2)

Parameters Required:

- User Code (eu_code)
- Continue Access Token:
 - Public key of the client instance (pk_ci)
 - Unique random value (as_cont_token_rand)
 - Resource Server Id (rs)
 - Resource Server Permission (rs_access)
 - Digital Signature:
 - Private Key of Authorization Server (invk pk_as)



Display User Code (3)

Parameters Required:

- User Code (eu_code)

Simulation Steps:

- Create a channel between the client instance and the end user having both confidentiality and authenticity properties to send the user code



Open URI (4), AuthN (5), Code (6), AuthZ (7), Completed (8)

Parameters Required:

- User Code (eu_code)

Simulation Steps:

- Assumption: User knows the correct URI and directly establishes a communication channel with the authorization server.



Continue Request (B) (11)

Parameters Required:

- Continue Access Token (from 2):
 - Public key of the client instance (pk_ci)
 - Unique random value (as_cont_token_rand)
 - Resource Server Id (rs)
 - Resource Server Permission (rs_access)
 - Digital Signature:
 - Private Key of Authorization Server (invk pk_as)
- Digital Signature:
 - Private Key of Client Instance (invk pk_ci)



Grant Access (12)

Parameters Required:

- Resource Server Permission (rs_access)
- Access Token:
 - Public key of the client instance (pk_ci)
 - Unique random value (as_token_rand)
 - Resource Server Id (rs)
 - Resource Server Permission (rs_access)
 - Digital Signature:
 - Private Key of Authorization Server (invk pk_as)



Access API (13)

- Access Token:
 - Public key of the client instance (pk_ci)
 - Unique random value (as_token_rand)
 - Resource Server Id (rs)
 - Resource Server Permission (rs_access)
 - Digital Signature:
 - Private Key of Authorization Server (invk pk_as)
- Digital Signature:
 - Private Key of Client Instance (invk pk_ci)



API Response (14)

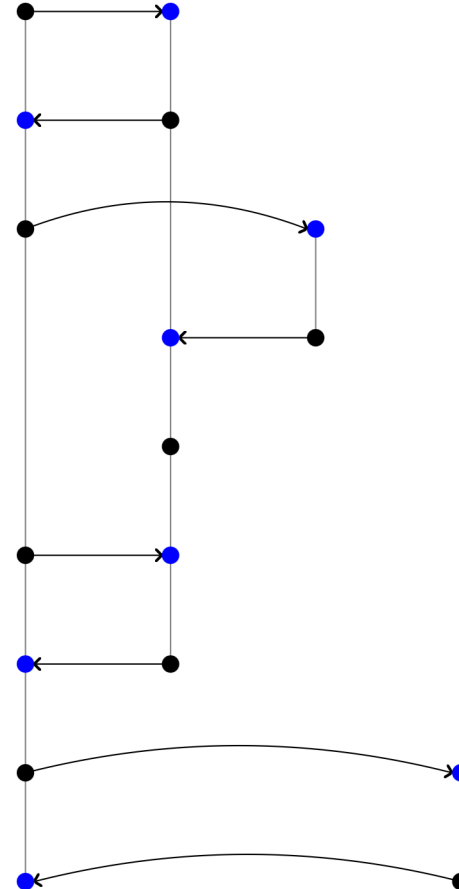
Parameters Required:

- Data (rs_data)
- Hash (Data + Access Token)

Analysis (Client Instance Perspective)

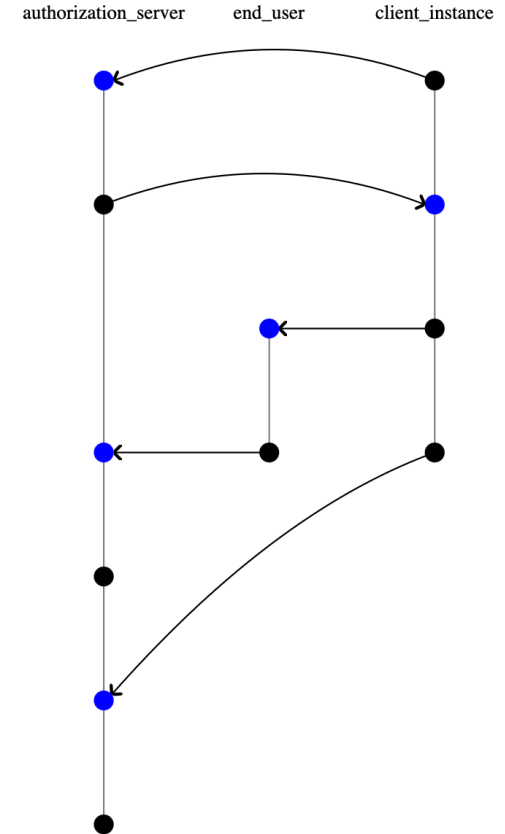
- = Transmission
- = Reception, explained
- = Reception, unexplained

client_instance authorization_server end_user resource_server



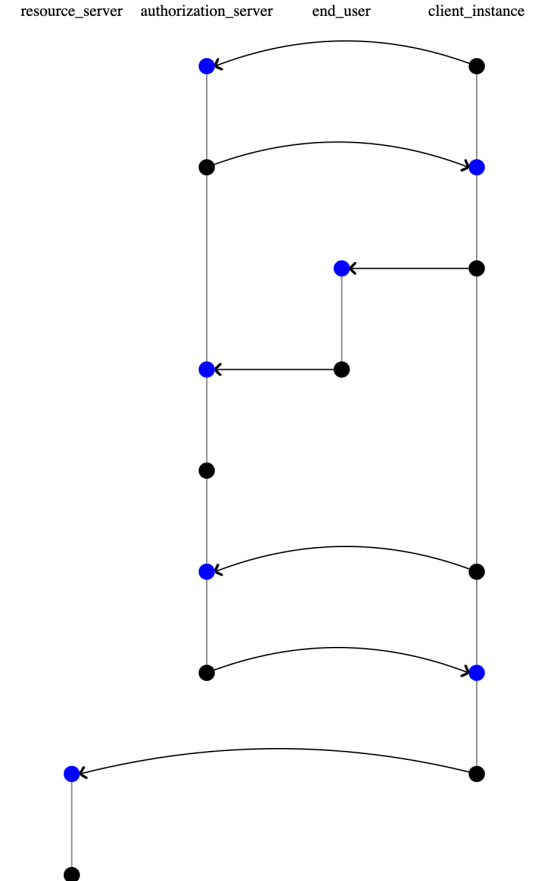
Analysis (Authorization Server Perspective)

- = Transmission
- = Reception, explained
- = Reception, unexplained



Analysis (Resource Server Perspective)

- = Transmission
- = Reception, explained
- = Reception, unexplained





Next Steps

- Many other shapes for the same model are due for analysis
- Only one flow of GNAP User-Code Interaction sequence is modelled
- Our existing model can be improved
- Many other sequences of GNAP needs to be modelled



Conclusion

- No critical security concerns found
- The GNAP protocol document can be improved by removing some ambiguity (Ex: which authentication protocol is used between end user and authorization server)
- Special thanks to Dr. Ed Zeigler and Enis Golaszewski!

Questions?